Who submitted the workload?

How to reproduce the tests and what is still missing | 16 October 2026

Enable it in the exporter

The PoC adds `submitted_by` to `karta_pod_workload_info` and `karta_workload_status`. Karta's CRD and SDK stay unchanged. Admission captures direct Kubernetes submissions. CEL reads the metadata supplied by each platform.

```
exporter:
  enabled: true
  useCatalog: true
  attribution:
    admission:
      enabled: true
    submittedByExpression: |
      (
        [
          "workload.dgxc.nvidia.com/submitter-id",
          "run.ai/submitted-by",
          "workflow.example.com/submitter",
          "platform.example.com/username",
          "platform.example.com/created-by"
        ]
        .filter(key, key in annotations && annotations[key] != "")
        .map(key, annotations[key])
        +
        [
          "workload.dgxc.nvidia.com/user",
          "identity.example.com/submitted-by",
          "osmo.submitted_by",
          "workflow.example.com/submitter",
          "platform.example.com/username",
          "platform.example.com/created-by"
        ]
        .filter(key, key in labels && labels[key] != "")
        .map(key, labels[key])
        + [""]
      )[0]
```

Admission records the authenticated Kubernetes caller before a selected workload is saved. Karta's operator serves this webhook over HTTPS (TLS). Within the selected scope, it also rejects changes to the recorded identity. Keep `operator.enabled` and `webhook.enabled` true. Setting `exporter.attribution.admission.enabled: false` stops capture; CEL can still read existing platform metadata.

The full expression above was tested. A workload label also supplies the identity on its Pod metric. An empty `submitted_by` means unknown; it is not a claim that no user exists. NVIDIA username and organization mapping still need agreement. The organization example below uses a test annotation; no organization metric was added.

Choose namespaces and workload types

Already supported: namespace include/exclude selectors and a resource allowlist. Add this under the existing exporter values; keep `submittedByExpression` unchanged.

```
exporter:
  attribution:
    admission:
      enabled: true
      namespaceSelector:
        matchExpressions:
          - key: karta.run.ai/capture-submitter
            operator: In
            values: ["true"]
          - key: kubernetes.io/metadata.name
            operator: NotIn
            values: [kube-system, karta-system]
      resourceRules:
        - apiGroups: ["apps"]
          apiVersions: ["v1"]
          resources: ["deployments", "statefulsets"]
        - apiGroups: ["batch"]
          apiVersions: ["v1"]
          resources: ["jobs", "cronjobs"]
        - apiGroups: [""]
          apiVersions: ["v1"]
          resources: ["pods"]
```

Include namespaces labeled `karta.run.ai/capture-submitter=true`; exclude the two named namespaces even if labeled. Both expressions must match. To include namespace names, use `kubernetes.io/metadata.name` with `In`.

Resource rules use API group/version and plural resource names: `apps/v1/deployments` selects Deployment. Today, exclude a type by removing it from this allowlist. Replace `karta-system` with the operator's namespace.

Proposed: match group, version and kind (GVK)

The chart does not expose `matchConditions` yet. Proposed addition under the same admission block, applied to both capture and update protection:

```
exporter:
  attribution:
    admission:
      matchConditions: # Proposed; not implemented in the chart
        - name: include-gvks
          expression: |
            request.kind.version == "v1" && (
              (request.kind.group == "apps" &&
               request.kind.kind in ["Deployment", "StatefulSet"]) ||
              (request.kind.group == "batch" &&
               request.kind.kind in ["Job", "CronJob"]) ||
              (request.kind.group == "" && request.kind.kind == "Pod"))
        - name: exclude-gvks
          expression: |
            !(request.kind.group == "batch" &&
              request.kind.version == "v1" &&
              request.kind.kind == "CronJob")
```

All conditions must pass, so the CronJob exclusion wins. Excluded requests skip submitter capture and update protection. `submittedByExpression` separately selects the identity to export. Namespace labels and scope settings must remain administrator-controlled.

This follows Kubernetes webhook matching. Only `matchConditions` is proposed; namespace selectors and resource allowlists already work.

Direct Kubernetes: capture and protect the caller

As an administrator, create `karta-label-direct` and label it `karta.run.ai/capture-submitter=true`. Use a user's kubeconfig with get/create/patch permission there. The test used a one-hour certificate for `alice@example.test`, without impersonation.

Save as `direct-job.yaml`:

```
apiVersion: batch/v1
kind: Job
metadata: {name: cel-label-job, namespace: karta-label-direct}
spec:
  backoffLimit: 0
  ttlSecondsAfterFinished: 3600
  template:
    spec:
      automountServiceAccountToken: false
      restartPolicy: Never
      containers:
      - name: work
        image: busybox:1.37.0
        command: [sh, -c, "echo submitter-demo; sleep 300"]
        resources:
          requests: {cpu: 5m, memory: 8Mi}
          limits: {cpu: 100m, memory: 32Mi}
```

```
kubectl --kubeconfig "$USER_KUBECONFIG" auth whoami
kubectl --kubeconfig "$USER_KUBECONFIG" apply -f direct-job.yaml
```

Admission wrote `metadata.annotations["workload.dgxc.nvidia.com/submitter-id"]` on the Job as `alice@example.test`. CEL reads that annotation; the Pod metric inherits it from the Job. The Pod needs no copied annotation. The Job completed after 300 seconds; a separate Deployment also ran successfully. Selected metric labels:

```
karta_workload_status{workload="cel-label-job",
  phase="Completed", submitted_by="alice@example.test"} 1
karta_pod_workload_info{workload="cel-label-job",
  submitted_by="alice@example.test"} 1
```

Try claiming another user at creation, then changing the saved identity:

```
KEY=workload.dgxc.nvidia.com/submitter-id
kubectl create --dry-run=client -f direct-job.yaml -o json |
  jq --arg key "$KEY" '.metadata.name="cel-label-forged" |
    .metadata.annotations[$key]="bob@example.test"' |
  kubectl --kubeconfig "$USER_KUBECONFIG" create -f -
kubectl --kubeconfig "$USER_KUBECONFIG" -n karta-label-direct \
  annotate job cel-label-job "$KEY=bob@example.test" --overwrite
```

Both were rejected. A caller may prefill their own matching identity. Unrelated metadata is preserved. Protection applies only to the configured scope; system-owned children are skipped. A service account alone does not identify the human behind it.

Captured requests and rejection results.

Run:AI: submit training and inference

Use a token with submission permission and set `RUNAI_API`, `RUNAI_TOKEN`, `PROJECT_ID` and `CLUSTER_ID`. The test used the Run:AI password login flow. These commands contain the full requests; no request files need to be downloaded.

```
jq -n --arg p "$PROJECT_ID" --arg c "$CLUSTER_ID" '{
  name: "cel-label-training", projectId: $p, clusterId: $c,
  spec: {
    image: "busybox:1.37.0", command: "sh",
    args: "-c \"echo attribution-start; sleep 300; echo done\"",
    backoffLimit: 0, restartPolicy: "Never",
    compute: {
      cpuCoreRequest: 0.05, cpuCoreLimit: 0.2,
      cpuMemoryRequest: "64Mi", cpuMemoryLimit: "128Mi",
      gpuDevicesRequest: 0
    }
  }
}' > training.json

jq '
  .name="cel-label-inference" |
  del(.spec.backoffLimit, .spec.restartPolicy) |
  .spec.args=("-c \"mkdir -p /tmp/www; " +
    "echo attribution-inference > /tmp/www/index.html; " +
    "httpd -f -p 8080 -h /tmp/www\"") |
  .spec.autoscaling={minReplicas:1,maxReplicas:1} |
  .spec.servingPort={
    container:8080,protocol:"http",clusterLocalAccessOnly:true}
' training.json > inference.json

for kind in training inference; do
  curl -fsS "$RUNAI_API/api/v1/workloads/${kind}s" \
    -H "Authorization: Bearer $RUNAI_TOKEN" \
    -H 'Content-Type: application/json' --data-binary @"$kind.json"
done
```

Training completed after 300 seconds. Inference served HTTP 200 on port 8080 with one replica. These CPU-only commands exercise submission and attribution, not model inference.

Required setup for this test:

- A Run:AI project and enough CPU capacity for both workloads.
- Knative serving for this inference path. Its `kubernetes.podspec-schedulername` and `kubernetes.podspec-affinity` flags allowed the scheduler and affinity settings; they do not enable identity extraction.
- Exporter RBAC with `get/list/watch` on `run.ai/workloadrunners`, so Karta can inspect the ownership graph. Existing access to Jobs, Services and Pods remains required.

Run:AI: where the identity appears

Run:AI supplied the identity on these objects. Captured fields:

```
apiVersion: batch/v1
kind: Job
metadata:
  name: cel-label-training
  annotations:
    run.ai/submitted-by: test@run.ai
---
apiVersion: serving.knative.dev/v1
kind: Service
metadata:
  name: cel-label-inference
  annotations:
    run.ai/submitted-by: test@run.ai
```

CEL reads `annotations["run.ai/submitted-by"]`. The WorkloadRunner CRs had no submitter annotation. Karta attributed these Pods to the Job and Service above. The inference Pod had no submitter key; its metric inherited the Service's value.

```
karta_workload_status{
  workload="cel-label-training", phase="Completed",
  submitted_by="test@run.ai"
} 1
karta_pod_workload_info{
  workload="cel-label-inference", submitted_by="test@run.ai"
} 1
```

The same CEL reads the annotation on both the Job and the Knative Service. No external identity mapper was used. The separate exporter was for lab isolation. HTTP was checked through a Knative revision proxy port-forward; external ingress was not tested.

Current gap: agree the NVIDIA username format with Run:AI. No submitter propagation bug was observed in these two paths.

See the captured resource fields and Run:AI metrics.

OSMO

Use a logged-in OSMO CLI. Replace `YOUR_CPU_PLATFORM` and `YOUR_POOL` with your platform and pool. Each block creates and submits the complete workflow; no files to download.

Single task:

```
cat > osmo-single.yaml <<'YAML'
workflow:
  name: cel-label-single
  resources:
    default:
      cpu: 1
      memory: 512Mi
      storage: 1Gi
      platform: YOUR_CPU_PLATFORM
  tasks:
  - name: worker
    image: ubuntu:24.04
    command: [/bin/sh, -c]
    args:
    - echo attribution-start; sleep 300; echo attribution-complete
YAML
osmo workflow submit osmo-single.yaml --pool YOUR_POOL
```

Leader/worker group:

```
cat > osmo-group.yaml <<'YAML'
workflow:
  name: cel-label-group
  resources:
    default:
      cpu: 1
      memory: 512Mi
      storage: 1Gi
      platform: YOUR_CPU_PLATFORM
  groups:
  - name: pair
    tasks:
    - name: leader
      image: ubuntu:24.04
      command: [/bin/sh, -c]
      args:
      - echo attribution-start; sleep 300; echo attribution-complete
      lead: true
    - name: worker
      image: ubuntu:24.04
      command: [/bin/sh, -c]
      args:
      - echo attribution-start; sleep 300; echo attribution-complete
YAML
osmo workflow submit osmo-group.yaml --pool YOUR_POOL
```

Use the workflow ID returned by each command to check completion:

```
osmo workflow query YOUR_WORKFLOW_ID
```

These are the same workflow specifications used in the API test below. The CLI commands are the documented reproduction path; the saved results came from REST submissions.

OSMO: result and existing options

OSMO 6.3.1 already supplies `osmo.submitted_by`. In the single-task and leader/worker tests, the API returned `submitted_by: alice@example.test`; task Pods and KAI PodGroups carried `osmo.submitted_by: alice`. Both workflows completed. In the repeated run, all three task exit codes were 0.

An administrator can add the agreed username label without changing OSMO code. Register this Pod template and append its name to the selected platform's `override_pod_template` list. Keep the existing templates.

```
metadata:
  labels:
    workload.dgxc.nvidia.com/user: "{{WF_SUBMITTED_BY | default('')}}"
    osmo.submitted_by: "{{WF_SUBMITTED_BY | default('')}}"
```

In 6.3.1 database configuration mode, create the named template through `PUT /api/configs/pod_template/attribution` with `{"configs": <template>}`; use an administrator credential. In ConfigMap mode, manage it through Helm/GitOps. The 6.3.1 service chart uses `services.configs.podTemplates` and `services.configs.pools`. Version 6.4 is ConfigMap-only: use `configuration.podTemplates` and `configuration.pools`. If `configuration.snapshot` is set, edit the equivalent fields inside that snapshot instead. Do not change configuration mode for this example.

Captured Pod fields, with unrelated metadata omitted:

```
metadata:
  labels:
    osmo.workflow_id: suggestion-shared-email-1
    osmo.task_name: worker
    osmo.submitted_by: alice
    workload.dgxc.nvidia.com/user: alice
```

```
karta_pod_workload_info{
  workload="21c6fd9fd8db4da8-bb22bddc61ac4673",
  workload_kind="Pod", submitted_by="alice"
} 1
```

The same value appeared on `karta_workload_status`. This is sufficient if `alice` is the agreed unique username. It is not a directory lookup. The template variable has already lost the email domain; `alice@example.test` and `alice@another.example` cannot be distinguished from that value alone.

Current edge case: the user `a` has no native submitter label. An unguarded `{{WF_SUBMITTED_BY}}` template failed and left the workflow PENDING. The guard above keeps missing attribution empty. The one-character omission and raw-principal limitation are included in the OSMO draft.

Version note: workflow `labels` / CLI `--label` arrived in 6.4.0. Those are caller-supplied values. If used, configure `workflow.labels_config.pod_label_prefix: caller.example.com/` in the appropriate configuration wrapper and keep identity templates administrator-owned. Do not use caller labels as proof of who submitted.

OSMO captures retain the original requests and outcomes. These live tests used development authentication headers; production login and NVIDIA username agreement remain unverified.

OSMO: select the identity and preserve the full value

For an authenticated 6.3.1 gateway, select the username claim on each human JWT provider. Keep existing issuer, audience, JWKS and other providers:

```
gateway:
  envoy:
    jwt:
      user_header: x-osmo-user
      providers:
        - issuer: https://idp.example.test/realms/osmo
          audience: YOUR_CLI_OR_BROWSER_AUDIENCE
          jwks_uri: https://idp.example.test/realms/osmo/certs
          cluster: idp
          user_claim: preferred_username
```

Use a verified, unique IdP claim for browser and CLI tokens. Keep `unique_name` for OSMO-issued tokens. For 6.4 external OIDC, use `authentication.externalOidc.userClaim`. Changing it changes the account identity; existing tokens and ownership do not migrate. This setting was source-reviewed, not tested with a production IdP.

A maintained administrator mapping is another existing option. Add this to the selected platform's `default_variables` and append the annotation to its Pod template:

```
# Platform default_variables
KARTA_USER_BY_SUBMITTER:
  alice:
    principal: alice@example.test
    org: demo_research_training
```

```
metadata:
  annotations:
    workload.dgxc.nvidia.com/submitter-id: >-
      {{KARTA_USER_BY_SUBMITTER.get(
        WF_SUBMITTED_BY | default(''), {}).get('principal', '')}}
    workload.dgxc.nvidia.com/org_hierarchy: >-
      {{KARTA_USER_BY_SUBMITTER.get(
        WF_SUBMITTED_BY | default(''), {}).get('org', '')}}
```

Both `alice` and `alice@example.test` produced `submitted_by="alice@example.test"` on the two Karta metrics. Missing `a` stayed empty. All three groups completed; leaders exited 0, workers had no reported exit code. This requires unique usernames and a maintained map; it cannot recover an ambiguous identity. Organization was a test value. No organization metric was added.

OSMO also already exports the full stored identity in its own metrics:

```
osmo_tasks_count{
  pool="default", status="RUNNING",
  user="alice@example.test",
  workflow_uuid="4c92f554382c4875b9e4add587563721"
} 1.0
```

Keep `services.service.disableTaskMetrics: false` and scrape port 9464, `/metrics`. This was enabled in the lab. The gauge counts tasks; completed-task coverage is short. A UUID join supports reporting, but does not populate Karta's metadata-only CEL input.

OSMO already has workflow/group UUIDs and KAI PodGroups. Karta currently follows controller ownership, so these bare task Pods remain individual workloads. That grouping gap belongs to Karta. The OSMO request is for lossless principal access. Source audit and retained captures document the settings and test limits.

NVCF / NVCT: reproduce the function

The container function served HTTP 200 and the finite task completed. Both carried account and workload IDs, but neither placed the authenticated caller in Kubernetes metadata. Both tested Karta exporter PoC metric families kept `submitted_by=""`.

Use your management URL/token as `NVCF_API` / `NVCF_ADMIN_TOKEN`, and an available `GPU` / `INSTANCE_TYPE`. The account needs function registration and deployment permission. These commands contain the complete HTTP fixture:

```
curl -fsS "$NVCF_API/v2/nvcf/functions" \
  -H "Authorization: Bearer $NVCF_ADMIN_TOKEN" \
  -H 'Content-Type: application/json' --data-binary @- \
  > function.json <<'JSON'
{
  "name":"cel-label-function",
  "containerImage":"nvcr.io/nvidia/omniverse/simple-nginx:1.0.2",
  "inferenceUrl":"/", "inferencePort":80,
  "health":{"protocol":"HTTP","uri":"/","port":80,
            "timeout":"PT10S","expectedStatusCode":200}
}
JSON
FID=$(jq -r '.function.id' function.json)
VID=$(jq -r '.function.versionId' function.json)
DEPLOY="$NVCF_API/v2/nvcf/deployments/functions/$FID/versions/$VID"
jq -n --arg gpu "$GPU" --arg it "$INSTANCE_TYPE" \
  '{deploymentSpecifications:[{gpu:$gpu,instanceType:$it,
    minInstances:1,maxInstances:1,maxRequestConcurrency:1}]}' \
  > deploy.json
curl -fsS "$DEPLOY" -H "Authorization: Bearer $NVCF_ADMIN_TOKEN" \
  -H 'Content-Type: application/json' --data-binary @deploy.json
```

Check the function version until ACTIVE, then invoke it using your invocation URL/domain and key. The tested gateway routes by the function Host header:

```
curl -fsS "$NVCF_API/v2/nvcf/functions/$FID/versions/$VID" \
  -H "Authorization: Bearer $NVCF_ADMIN_TOKEN"
curl -fsS "$NVCF_INVOKE_URL/" \
  -H "Host: $FID.$NVCF_INVOKE_DOMAIN" \
  -H "Authorization: Bearer $NVCF_API_KEY"
# Release this function's allocation before the finite task.
curl -fsS -X DELETE "$DEPLOY" \
  -H "Authorization: Bearer $NVCF_ADMIN_TOKEN"
```

Observed: ACTIVE, HTTP 200. The follow-up added `{"key":"KARTA_TEST_SUBMITTED_BY","value":"alice@example.test"}` to `containerEnvironment`. It reached the function's Pod spec, but no annotation or label. The image had no shell, so its process environment was not inspected.

Lab versions: NVCF `1.18.0`, NVCT `1.66.0`, NVCA `3.10.0`, ReVal `0.20.2`. The lab used service credentials, fake GPU allocation and an HTTP fixture. The two test Pods needed a lab-only CPU resize to 100m; the API requests do not apply that adjustment. Human login, hosted NVCF and model inference were not tested.

Original requests and outcomes and environment follow-up.

NVCT: reproduce a task that finishes

Use `NVCT_API` / `NVCT_API_KEY` with task-launch and task-detail permissions. A bare sleep does not complete the NVCT protocol: write progress, reach 100, then exit. This script reproduces the saved finite task, with result uploads disabled:

```
cat > nvct-task.sh <<'SH'
set -eu
for attempt in $(seq 1 60); do
  [ -d "$NVCT_RESULTS_DIR" ] && break
  sleep 1
done
[ -d "$NVCT_RESULTS_DIR" ]
sleep 1
progress() {
  printf '{"taskId":"%s","percentComplete":%s,"name":"%s","lastUpdatedAt":"%s"}\n' \
    "$NVCT_TASK_ID" "$1" "$2" "$(date -u +%Y-%m-%dT%H:%M:%SZ)" \
    > "$NVCT_PROGRESS_FILE_PATH.tmp"
  mv "$NVCT_PROGRESS_FILE_PATH.tmp" "$NVCT_PROGRESS_FILE_PATH"
}
progress 1 running
sleep 180
progress 100 completed
sleep 10
SH
jq -n --arg gpu "$GPU" --arg it "$INSTANCE_TYPE" \
  --arg script "$(base64 < nvct-task.sh | tr -d '\n')" '{
  name:"cel-label-task", gpuSpecification:{gpu:$gpu,instanceType:$it},
  containerImage:"nvcr.io/nvidia/cuda:12.6.3-base-ubuntu22.04",
  containerArgs:"bash -c '\''printf %s \"$ATTRIBUTION_SCRIPT\" | base64 -d | bash'\''",
  maxRuntimeDuration:"PT5M", maxQueuedDuration:"PT10M",
  terminationGracePeriodDuration:"PT15S", resultHandlingStrategy:"NONE",
  containerEnvironment:[{key:"ATTRIBUTION_SCRIPT",value:$script}]
}' > task.json
curl -fsS "$NVCT_API/v1/nvct/tasks" \
  -H "Authorization: Bearer $NVCT_API_KEY" \
  -H 'Content-Type: application/json' --data-binary @task.json > task-result.json
TASK_ID=$(jq -r '.task.id' task-result.json)
curl -fsS "$NVCT_API/v1/nvct/tasks/$TASK_ID" \
  -H "Authorization: Bearer $NVCT_API_KEY"
```

Observed: COMPLETED, 100%. The follow-up environment value also reached `printenv` in the task. Neither test projected it into metadata.

The lab needed these helper-image overrides; the public image references were confirmed in its current ConfigMap. Merge these settings into the existing values, preserving other sidecars:

```
nvctApi:
  remoteConfig:
    enabled: true
    configData:
      nvct:
        sidecars:
          init-container: "nvcr.io/nvidia/nvcf/nvcf-worker-init-oss:1.2.1"
          utils-container: "nvcr.io/nvidia/nvcf/nvcf-worker-task-oss:1.1.4"
```

These are the actual chart setting names. The application properties are `nvct.sidecars.init-container` and `nvct.sidecars.utils-container`. Earlier malformed-task cancellation left an ICMSRequest requiring manual cleanup. This and helper-image packaging are separate from the identity request; suggest separate issues if reproduced on the target release.

NVCF / NVCT: existing options and remaining gap

The tested Pods had `nca-id`, function/version or task IDs and an ICMSRequest owner. No human submitter appeared in labels or annotations. Captured metrics, with other labels omitted:

```
karta_pod_workload_info{workload_kind="Pod",submitted_by=""} 1
karta_workload_status{workload_kind="Pod",phase="Running",submitted_by=""} 1
```

Source review found these existing options:

- Audit joins: retain INFO-level `[AUDIT]` events from NVCF/NVCT. Join account plus function/version/deployment or task ID, preserving operation and time. Events include authenticated actor/subject identity. A shared service credential remains a service identity. This needs a log pipeline and identity mapping; it does not feed Karta's metadata-only CEL. Audit emission.
- Static classification: NVCA `agent.customAnnotations` populates `nvca-namespace-pod-annotations` and is copied literally onto directly created worker Pods. It can identify a dedicated pool's organization; it cannot derive each submitter. `worker.featureFlags: ["HelmCustomAnnotations"]` enables the separate Helm utils-Pod copy, not a general chart-Pod stamp. Workload application.
- Helm metadata: function `deploymentSpecifications[].configuration` and task `gpuSpecification.configuration` already reach chart values. A chart can stamp root and Pod-template annotations without NVCF source changes. For a chart supporting these keys, the configuration is:

```
{
  "commonAnnotations": {"workload.dgxc.nvidia.com/submitter-id":"alice@example.test"},
  "podAnnotations": {"workload.dgxc.nvidia.com/submitter-id":"alice@example.test"}
}
```

The pinned public nginx chart `18.2.5` rendered both locations locally; its `commonAnnotations` and `podAnnotations` keys are chart-specific. The lab ReVal had `reval.serviceConfig.skipSanitizeObjectMetadata: true`. If sanitization is enabled, retain the root key through `reval.serviceConfig.preserveAnnotations: [workload.dgxc.nvidia.com/submitter-id]`. These are exact keys, not regex; `preserveLabels` alone does not preserve Deployment template labels. ReVal settings.

Full live Helm submission stopped before creation: `registry-1.docker.io` was not a recognized HELM registry; `docker.io` required a registered HELM credential. An account credential needs `manage_registry_credentials` (or `apikey:manage_registry_credentials`) to inspect `GET /v2/nvcf/registry-credentials?artifactType=HELM` and register real read credentials through `POST /v2/nvcf/registry-credentials`, with `registryHostname: docker.io` and `artifactTypes: [HELM]`. Exact API authorization and fields. No credential was added or shared setting changed; the follow-up did not execute. No full Helm-to-Karta success is claimed.

Helm values and environment variables are caller-supplied. A trusted upstream submitter can supply a verified principal, but the submission path must enforce that trust. The remaining request is automatic capture of the authenticated function creator/deployer and task submitter, then supported workload/Pod metadata propagation. Invokers are separate.

Grouping exists through account, function/version, deployment, task, ICMSRequest and MiniService IDs. Standard Helm Deployment/Job owner chains also exist. In the direct-container captures, Pod owner references lacked `controller: true`, so the tested Karta exporter PoC selected each Pod. That is an integration limitation; stamping only ICMSRequest would not cover it. See the narrow feature-request draft.